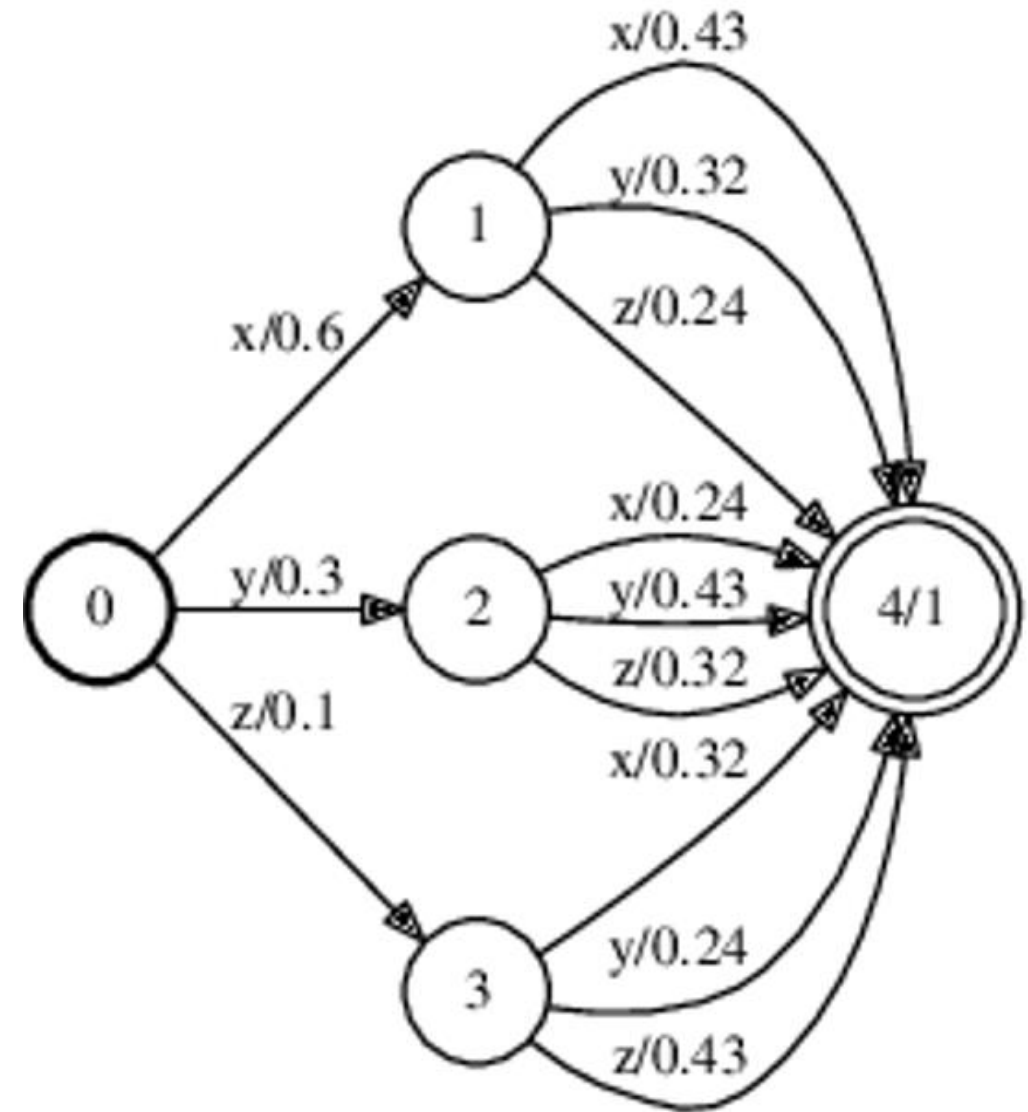
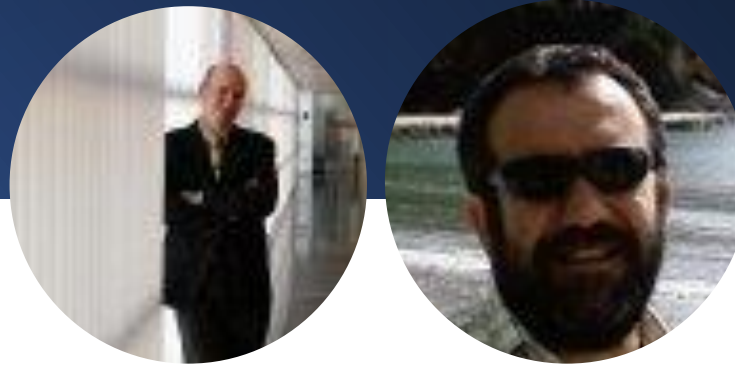


Learning Stochastic Regular Grammars by Means of a State Merging Method

By Sebastian David Albu



Well actually by these two guys here:



Rafael C. Carrasco

Jose Oncina

Departamento de Tecnología Informática y Computación

Universidad de Alicante, E-03071 Alicante

E-mail: (carrasco, joncina)@dtic.ua.es

What is this paper about?

Abstract

We propose a new algorithm which allows for the identification of any stochastic deterministic regular language as well as the determination of the probabilities of the strings in the language. The algorithm builds the prefix tree acceptor from the sample set and merges systematically equivalent states. Experimentally, it proves very fast and the time needed grows only linearly with the size of the sample set.

Hmm?

Abstract

We propose a new algorithm which allows for the identification of any **stochastic deterministic regular language** as well as the determination of the probabilities of the strings in the language. The algorithm builds the prefix tree acceptor from the sample set and merges systematically equivalent states. Experimentally, it proves very fast and the time needed grows only linearly with the size of the sample set.

Hmm?

Abstract

We propose a new algorithm which allows for the identification of any stochastic deterministic regular language as well as the determination of the probabilities of the strings in the language. The algorithm builds the **prefix tree acceptor** from the sample set and merges systematically equivalent states. Experimentally, it proves very fast and the time needed grows only linearly with the size of the sample set.

Hmm?

Abstract

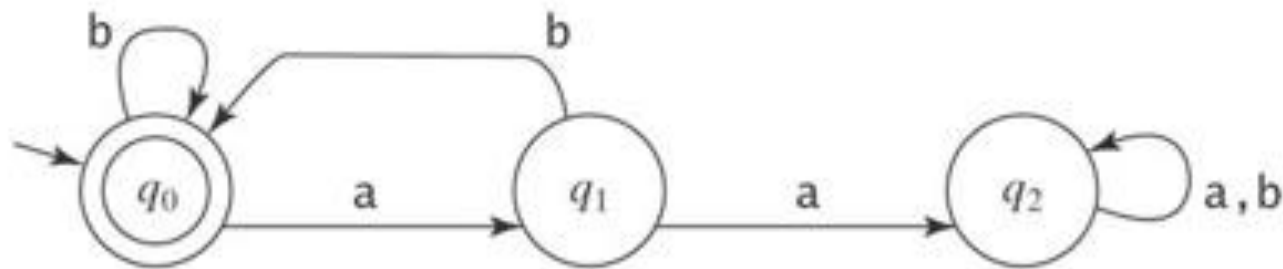
We propose a new algorithm which allows for the identification of any stochastic deterministic regular language as well as the determination of the probabilities of the strings in the language. The algorithm builds the prefix tree acceptor from the sample set and merges systematically equivalent states. Experimentally, it proves very fast and the time needed grows only linearly with the size of the sample set.

This might be clearer: Regular Languages

A language is **regular** iff it is accepted by some DFA.

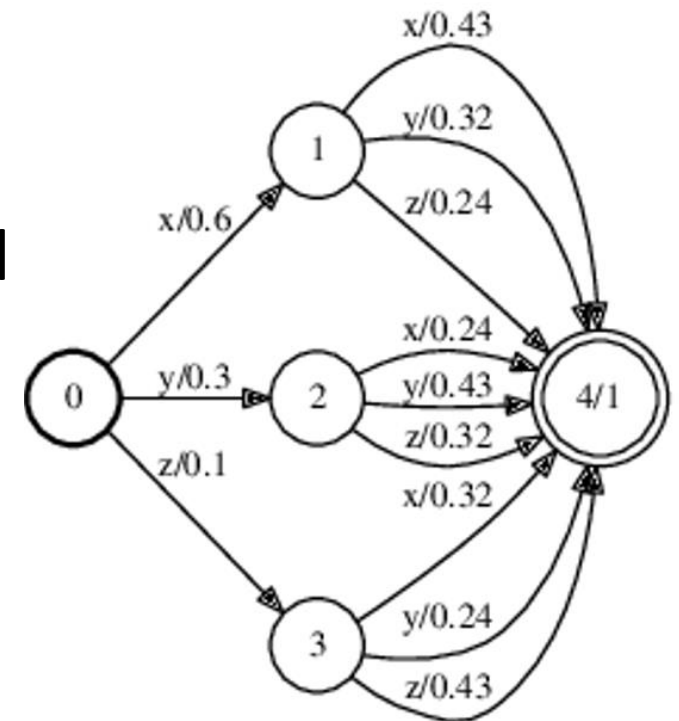
Example: $L = \{w \in \{a, b\}^* :$

every a is immediately followed by a $b\}$.



Now let us “add randomness”:

- Motivation: Most of realistic situations involve examples provided by a random source.
- It has been proven already by prior research that identifying a grammar with only positive samples is possible.
- Since this proof has existed, there have been several attempts to learn languages that are based on stochastic decisions, neural networks and the hidden Markov models.
(You can only observe the emissions or output)
→ Basically, Black Box Testing 😊



Stochastic Regular Languages (SRL):

Let $\mathcal{A}$ be a finite alphabet, $\mathcal{A}^*$ the set of all strings on $\mathcal{A}$ and λ the empty string such that for every symbol a in $\mathcal{A}$ satisfies $a\lambda = \lambda a = a$. If w and x are strings of symbols in $\mathcal{A}^*$ and $w = xa$, then we will also write $x = wa^{-1}$. A *stochastic finite automaton* (SFA), $A = (\mathcal{A}, Q, P, q_1)$, consists of an alphabet $\mathcal{A}$, a finite set of nodes $Q = \{q_1, q_2, \dots, q_n\}$, with q_1 the initial node, and a set P of probability matrices $p_{ij}(a)$ giving the probability of a transition from node q_i to node q_j led by the symbol a in the alphabet. If we call p_{if} the probability that the string ends at node q_i , the following constraint applies:

$$p_{if} + \sum_{q_j \in Q} \sum_{a \in \mathcal{A}} p_{ij}(a) = 1. \quad (1)$$

Stochastic Regular Languages (SRL) and Stochastic Finite Automatons (SFA)

The probability $p(w)$ for the string w to be generated by A is defined by:

$$\begin{aligned} p(w) &= \sum_{q_j \in Q} p_{1j}(w) p_{jf} \\ p_{ij}(w) &= \sum_{q_k \in Q} \sum_{a \in \mathcal{A}} p_{ik}(wa^{-1}) p_{kj}(a) \end{aligned} \quad (2)$$

and the language generated by the automaton A is defined as:

$$L = \{w \in \mathcal{A}^* : p(w) \neq 0\} \quad (3)$$

Two SRL L_1 and L_2 are equivalent iff

$$L_1 \equiv L_2 \iff p_1(w) = p_2(w) \quad \forall w \in \mathcal{A}^*.$$

How do we get these probabilities?

The probability $p(w)$ for the string w to be generated by A is defined by:

$$\begin{aligned} p(w) &= \sum_{q_j \in Q} p_{1j}(w) p_{jf} \\ p_{ij}(w) &= \sum_{q_k \in Q} \sum_{a \in \mathcal{A}} p_{ik}(wa^{-1}) p_{kj}(a) \end{aligned} \quad (2)$$

and the language generated by the automaton A is defined as:

$$L = \{w \in \mathcal{A}^* : p(w) \neq 0\} \quad (3)$$

- Mathematically, if you have a sequence of states and transitions that lead to the generation of a string, say "abc," you would calculate the probability as:
- $P(\text{"abc"}) = P(\text{State1} \rightarrow \text{State2}) * P(\text{State2} \rightarrow \text{State3})$
 $* \dots * P(\text{StateN-1} \rightarrow \text{StateN})$ (but also applying Markov property)

Stochastic Regular Languages (SRL) and Stochastic Finite Automata (SFA)

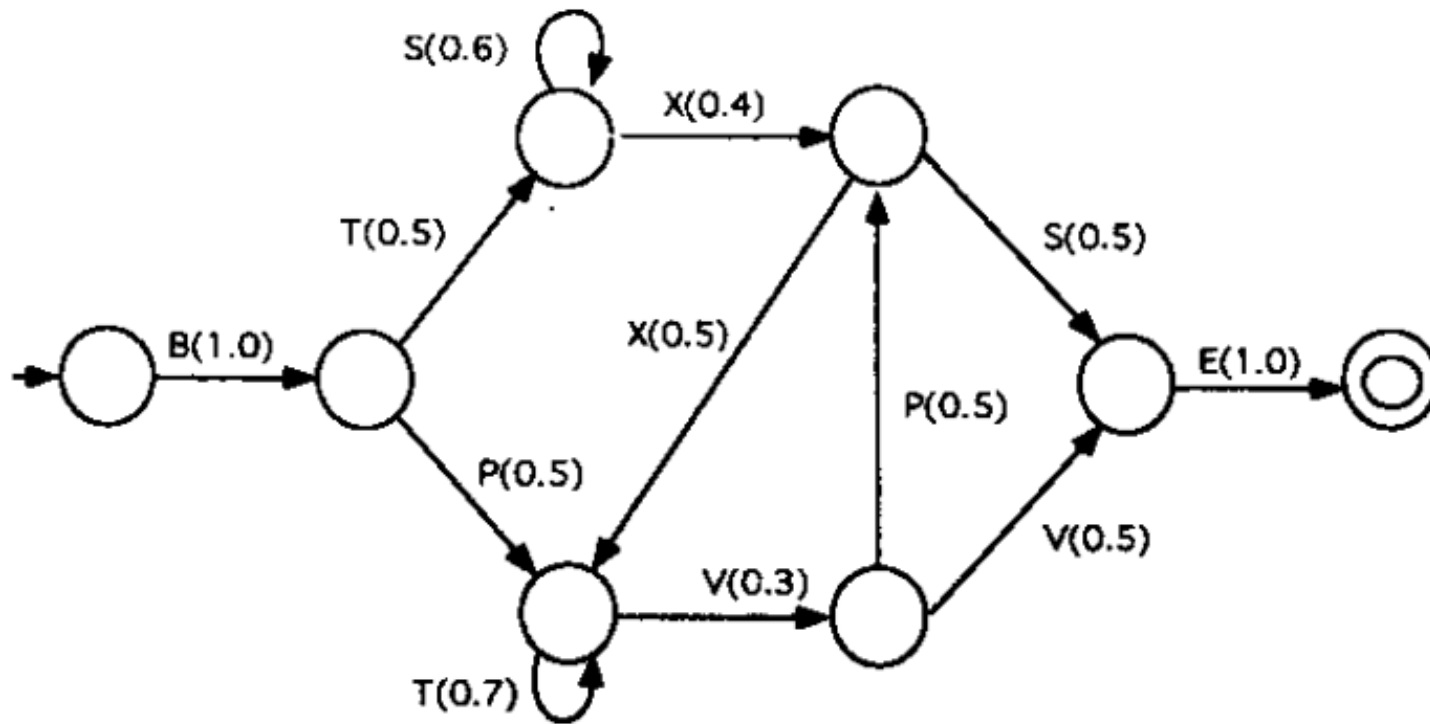
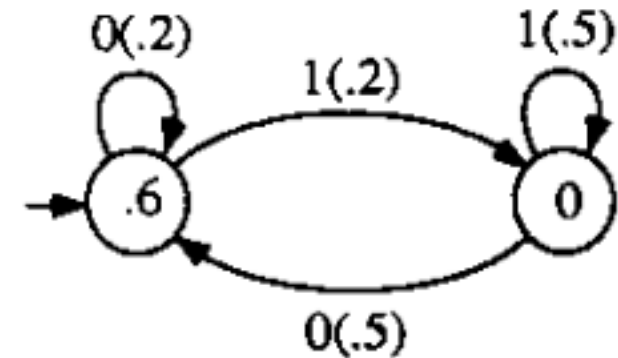


Figure 5: SFA corresponding to the Reber grammar

"Reber"	"Non-Reber"
BTSSXXTVWE	BTSSPXSE
BPVWE	BPTVWB
BTXXVPSE	BTXXWSE
BPVPXVPXVPXWE	BPVSPSE
BTSXXVPSE	BTSSE

Stochastic Regular Languages (SRL) and Stochastic Finite Automatons (SFA)

We will use the following example from now on:



We assume that the SFA outputs the following: (λ is an empty input)

$$S = \{110, \lambda, \lambda, \lambda, 0, \lambda, 00, 00, \lambda, \lambda, \lambda, 10110, \lambda, \lambda, 100\}.$$

Identifying regular languages

- A complete sample S consistent with L consists of two subsets S_+ with strings in L and S_- with strings not in L . The algorithm then identifies if adding new examples to S may only produce a finite number of changes of hypothesis.
- But SRL do not have the S_- property?
- Yes, BUT:
the statistical regularity is able to compensate for the lack of data. It has already been proven, that many recursively numberable sets of distribution and in particular SRL when probabilities are restricted to rationals are identifiable with probability 1, by means of enumerative algorithms.

Identifying stochastic regular languages and getting a stochastic DFA:

```
algorithm ALERGIA
input:
     $S$  : sample set of strings
     $\alpha$  : 1-confidence level
output:
    stochastic DFA
begin
     $A$  = stochastic Prefix Tree Acceptor from  $S$ 
    do (for  $j$  = successor( first node( $A$ )) to lastnode( $A$ ))
        do(for  $i$  = firstnode( $A$ ) to  $j$ )
            if compatible( $i, j$ )
                merge( $A, i, j$ )
                determinize( $A$ )
                exit ( $i$ -loop)
            end if
        end for
    end for
    return  $A$ 
end algorithm
```

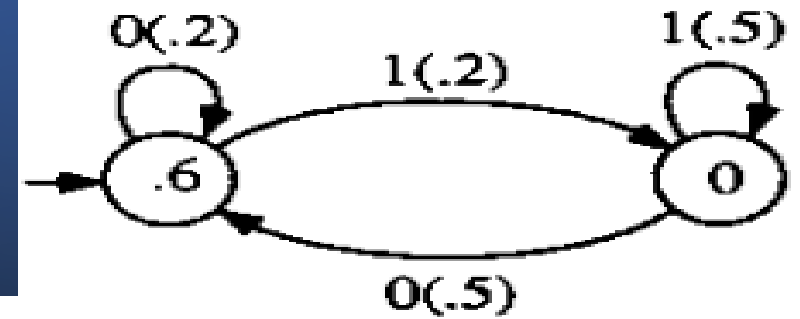
Identifying stochastic regular languages and getting a stochastic DFA:

```
algorithm ALERGIA
input:
     $S$  : sample set of strings
     $\alpha$  : 1-confidence level
output:
    stochastic DFA
begin
     $A$  = stochastic Prefix Tree Acceptor from  $S$ 
    do (for  $j$  = successor( first node( $A$ )) to lastnode( $A$ ))
        do(for  $i$  = firstnode( $A$ ) to  $j$ )
            if compatible( $i, j$ )
                merge( $A, i, j$ )
                determinize( $A$ )
                exit (i-loop)
            end if
        end for
    end for
    return  $A$ 
end algorithm
```

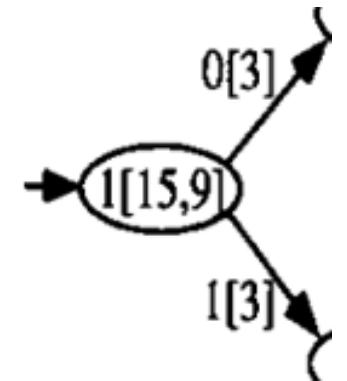
The (stochastic) prefix tree acceptor

- A stochastic prefix tree is a data structure used to represent and model sequences of events, where each event has a probabilistic outcome, and the tree structure captures the likelihood of different sequences of events
- A regular PTA: a tree shaped DFA that accepts precisely the positive examples and subsequently it tries to merge pairs of states in the canonical order of words (Lexicographic)

The (stochastic) prefix tree acceptor



- $S = \{110, \lambda, \lambda, \lambda, 0, \lambda, 00, 00, \lambda, \lambda, \lambda, 10110, \lambda, \lambda, 100\}$
- First number in brackets is number of strings arriving at that node
- 15 strings $\rightarrow$ so we have [15,] at start node
- Second number is amount of strings terminating in that node
- λ is an empty input, we have λ nine times $\rightarrow$ so we have [15, 9] in the start node. Looking at the SFA we have .6 for empty input and 60% of 15 = 9.
- We continue this procedure for every following node.
- The number before the brackets is the index
- The transitions are labeled like value[amount] $\rightarrow$ 0[3] etc.



The (stochastic) prefix tree acceptor

- For our example we will get the following PTA:

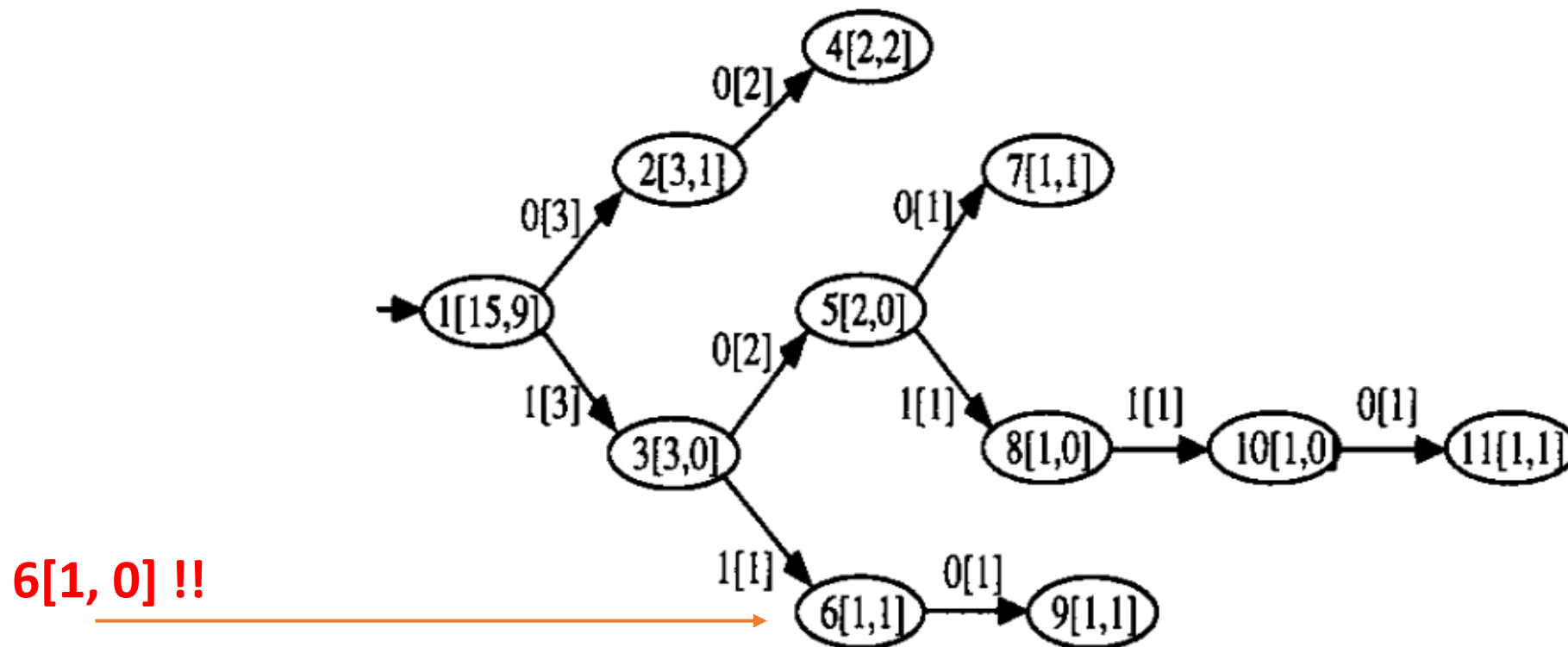


Figure 9: Prefix tree acceptor for the sample $S = \{110, \lambda, \lambda, \lambda, 0, \lambda, 00, 00, \lambda, \lambda, \lambda, 10110, \lambda, \lambda, 100\}$.

Identifying stochastic regular languages and getting a stochastic DFA:

```
algorithm ALERGIA
input:
     $S$  : sample set of strings
     $\alpha$  : 1-confidence level
output:
    stochastic DFA
begin
     $A$  = stochastic Prefix Tree Acceptor from  $S$ 
    do (for  $j$  = successor( first node( $A$ )) to lastnode( $A$ ))
        do(for  $i$  = firstnode( $A$ ) to  $j$ )
            if compatible( $i, j$ )
                merge( $A, i, j$ )
                determinize( $A$ )
                exit ( $i$ -loop)
            end if
        end for
    end for
    return  $A$ 
end algorithm
```

Merging two nodes

- If we have a partition x of the set Q of nodes of T , then $x(T)$ is the automaton obtained by merging the nodes in the same block of the partition. For example, in the given DFA, the canonical acceptor may be obtained by the prefix tree acceptor merging states labelled with numbers in the same block of the partition $\rightarrow x' = \{\{1,2,4,6,7\}, \{3,5\}\}$

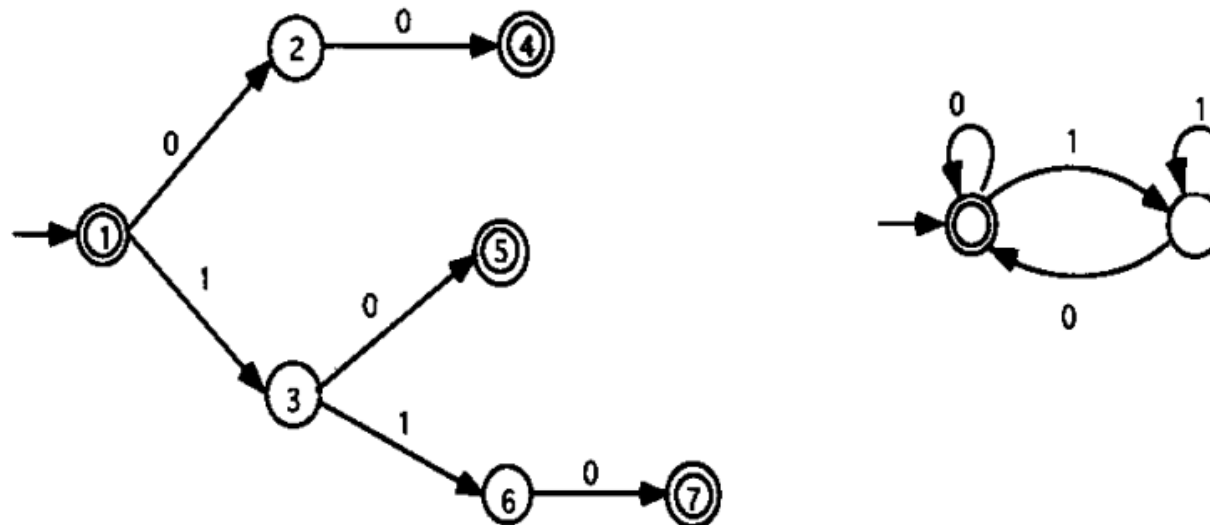


Figure 1: A prefix tree and the canonical acceptor for the regular set $(0 + 1)^*0$. Final states are doubled circled. Numbering of nodes follows the lexicographic order

Identifying stochastic regular languages and getting a stochastic DFA:

```
algorithm ALERGIA
input:
     $S$  : sample set of strings
     $\alpha$  : 1-confidence level
output:
    stochastic DFA
begin
     $A$  = stochastic Prefix Tree Acceptor from  $S$ 
    do (for  $j$  = successor( first node( $A$ )) to lastnode( $A$ ))
        do(for  $i$  = firstnode( $A$ ) to  $j$ )
            if compatible( $i, j$ )
                merge( $A, i, j$ )
                determinize( $A$ )
                exit ( $i$ -loop)
            end if
        end for
    end for
    return  $A$ 
end algorithm
```

Finding compatible nodes to merge

- This recursion also involves only a finite number of calls, due to the order of the merging process.
- When q_i and q_j are compared, q_j is always the root of a subtree of the PTA and therefore the language it generates is finite.
- So, there is no loop in the q_j subtree.

```
algorithm compatible
input:
     $i, j$  : nodes
output:
    boolean
begin
    if different( $n_i, f_i(\#), n_j, f_j(\#)$ )
        return false
    endif
    do( $\forall a \in \mathcal{A}$ )
        if different( $n_i, f_i(a), n_j, f_j(a)$ )
            return false
        end if
        if not compatible( $\delta(i, a), \delta(j, a)$ )
            return false
        end if
    end do
    return true
end algorithm
```

Finding compatible nodes to merge

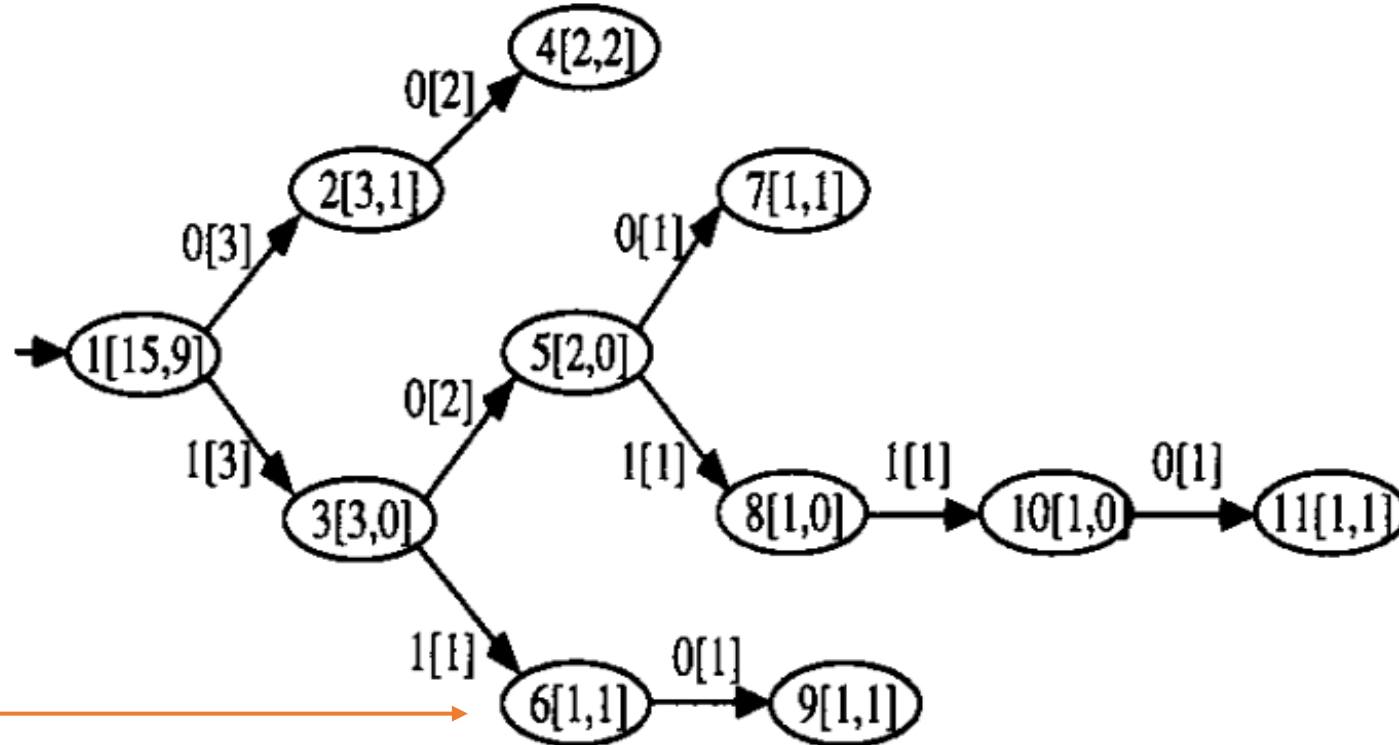
```
algorithm compatible
input:
    i, j : nodes
output:
    boolean
begin
    if different(ni, fi(#), nj, fj(#))
        return false
    endif
    do(∀a ∈ A)
        if different(ni, fi(a), nj, fj(a))
            return false
        end if
        if not compatible(δ(i, a), δ(j, a))
            return false
        end if
    end do
    return true
end algorithm
```

```
algorithm different
input:
    n, n': number of strings arriving at each node
    f, f': number of strings ending/following a given arc
output:
    boolean
begin
    return  $\left| \frac{f}{n} - \frac{f'}{n'} \right| > \sqrt{\frac{1}{2} \log \frac{2}{\alpha} \left( \frac{1}{\sqrt{n}} + \frac{1}{\sqrt{n'}} \right)}$ 
end algorithm
```

Keyword here is the „Hoeffding Bound“ [14]

Revisiting our PTA

- For our example we will get the following PTA:



6[1, 0] !!

Figure 9: Prefix tree acceptor for the sample $S = \{ 110, \lambda, \lambda, \lambda, 0, \lambda, 00, 00, \lambda, \lambda, \lambda, 10110, \lambda, \lambda, 100 \}$.

Finding compatible nodes to merge

$$\left| \frac{t}{n} - \frac{t'}{n'} \right| > \sqrt{\frac{1}{2} \log \frac{2}{\alpha}} \left(\frac{1}{\sqrt{n}} + \frac{1}{\sqrt{n'}} \right)$$

Only for illustration purposes, the parameter α will be arbitrarily set to 0.8. With this choice one has:

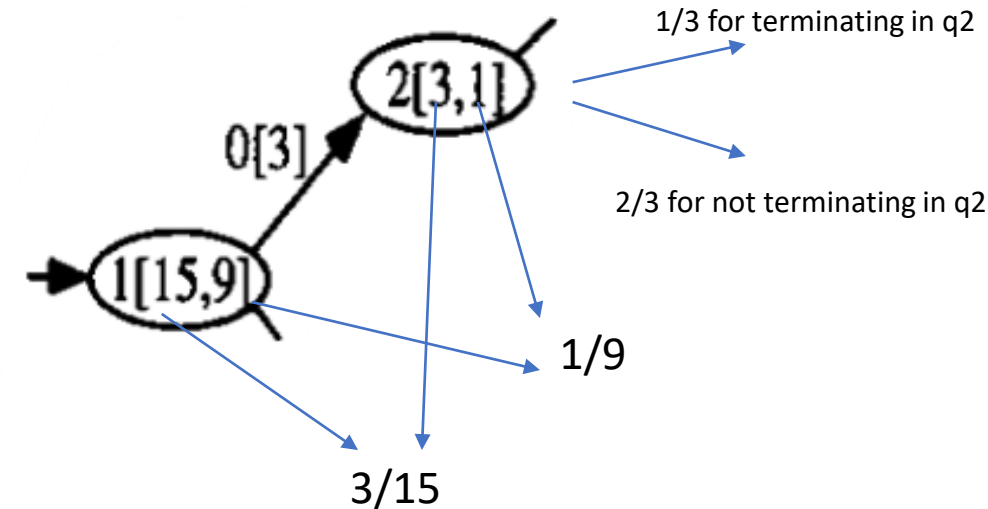
$$\gamma := \sqrt{\frac{1}{2} \log \frac{2}{\alpha}} \simeq 0.67$$

Now we want to merge q2 and q1 (labelled as 2 and 1). We compare the probabilities coming from the nodes. The termination nodes are found to be similar:

$$|p - p'| = \left| \frac{1}{3} - \frac{1}{9} \right| \simeq 0.26 < \gamma \left(\frac{1}{\sqrt{n}} + \frac{1}{\sqrt{n'}} \right) \simeq 0.55$$

Also, the outgoing transitions have similar probabilities:

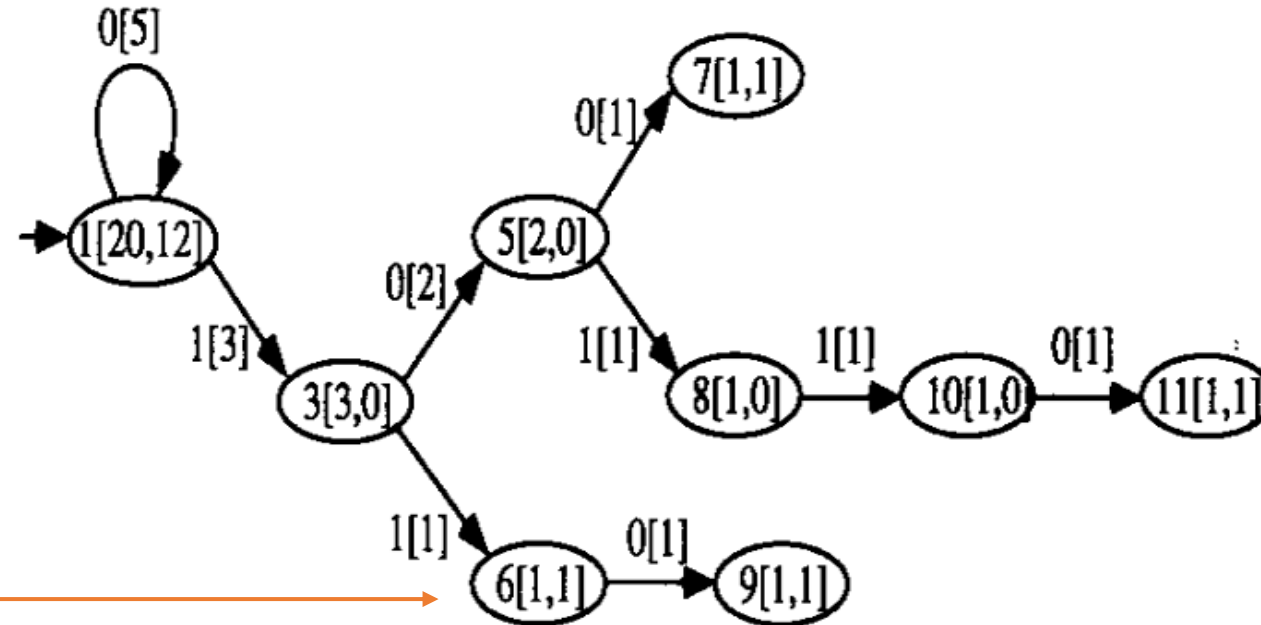
$$|p - p'| = \left| \frac{2}{3} - \frac{3}{15} \right| \simeq 0.46 < 0.55$$



Additionally, destination nodes need to be equivalent (q4 and q2) – which is the case, so we can merge the nodes (For the remaining nodes and merging states procedure all values will be calculated the same)

Finding compatible nodes to merge

$$\left| \frac{f}{n} - \frac{f'}{n'} \right| > \sqrt{\frac{1}{2} \log \frac{2}{\alpha}} \left(\frac{1}{\sqrt{n}} + \frac{1}{\sqrt{n'}} \right)$$



6[1, 0] !!

Figure 10: Prefix tree acceptor after merging q_2 and q_1 .

Now we want to merge q_1 and q_3 but the merge is rejected $\rightarrow$ probabilities 0.63 and 0.0 are larger than 0.53
 But the merge for q_5 and q_1 gives then an acceptance for $q_7 = q_1$, $q_8 = q_3$, $q_{10} = q_6$, $q_{11} = q_9$. So we get:

Finding compatible nodes to merge

$$\left| \frac{f}{n} - \frac{f'}{n'} \right| > \sqrt{\frac{1}{2} \log \frac{2}{\alpha}} \left(\frac{1}{\sqrt{n}} + \frac{1}{\sqrt{n'}} \right)$$

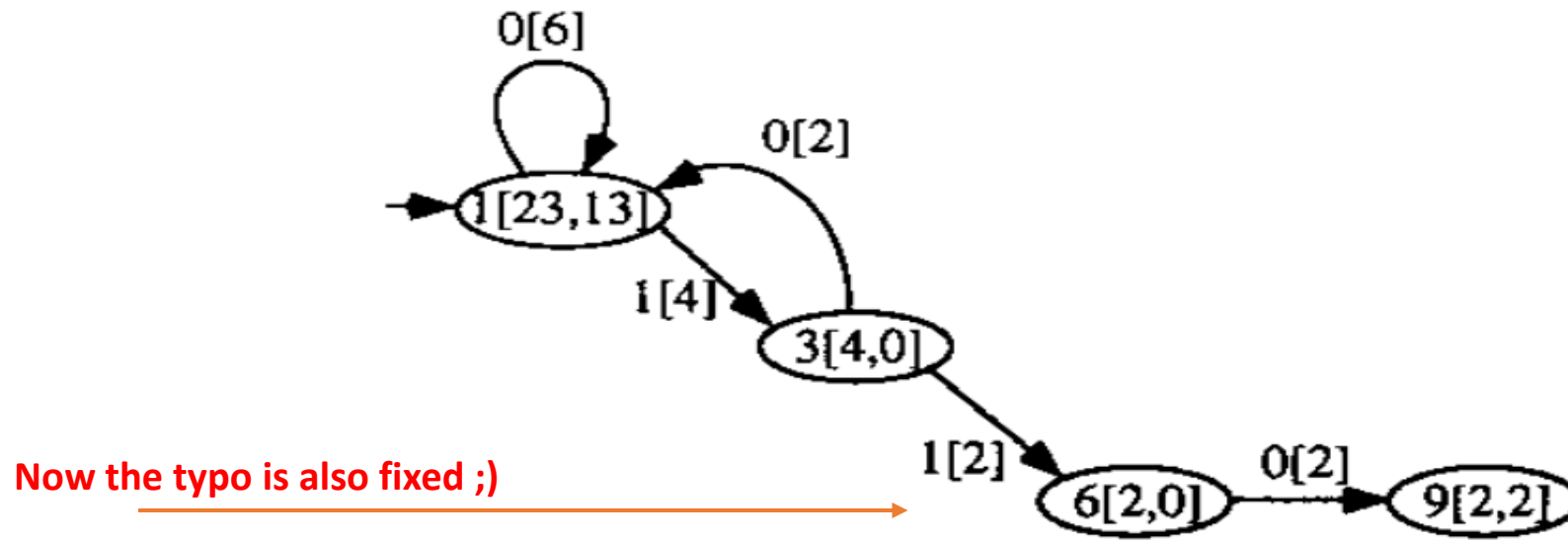


Figure 11: Automaton after merging q_5 and q_1 .

The merging of nodes q_6 and q_1 is rejected due to the difference in the transition probabilities with label 0 $\rightarrow$ 0.75 is larger than 0.61. Instead $q_6 = q_3$ will be accepted and the algorithm terminates:

Final automaton by running ALERGIA:

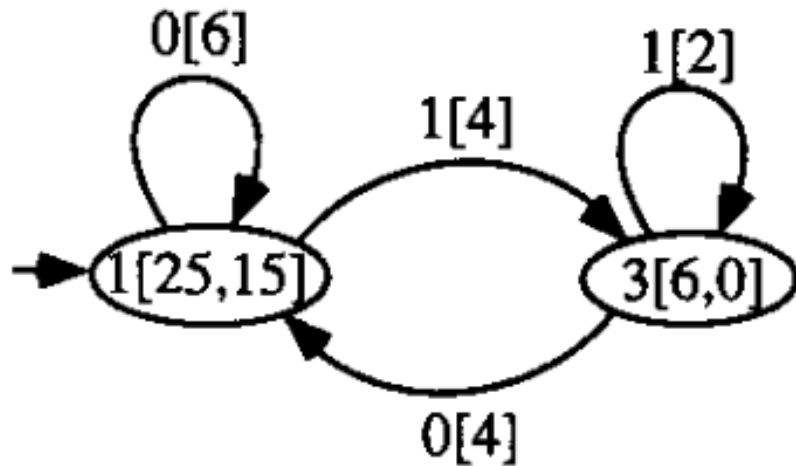


Figure 12: Automaton after merging q_6 and q_3 .

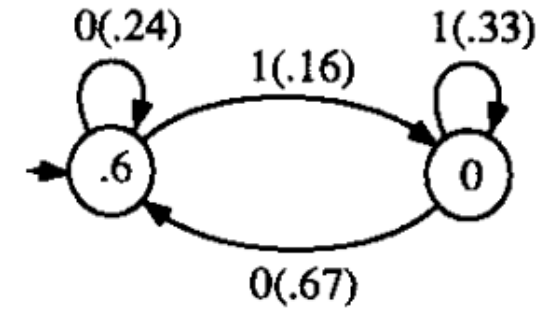


Figure 13: The final output.

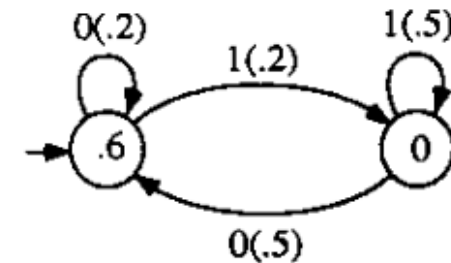


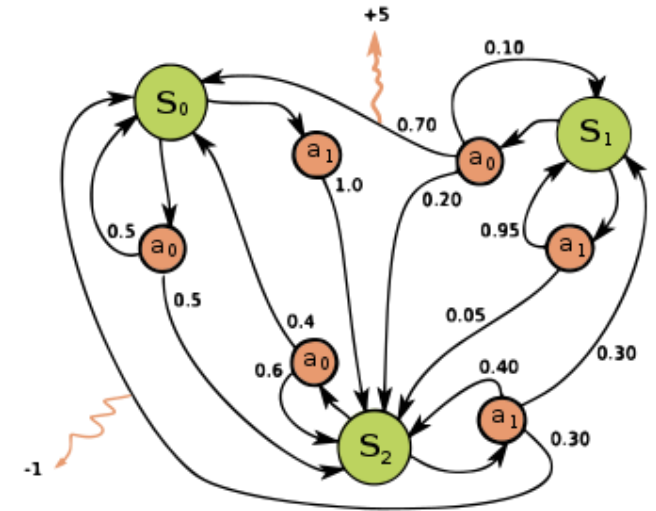
Figure 8: Stochastic finite automaton

We can see that the final learned SFA is not identical to the original SRL but we were able to approximate it quite decently and inside the wanted tolerance interval (we could set it empirically)

Application of this technique (in MBT):

- Learning Probabilistic Systems for Model Checking
- Randomized Testing and learning CFGs
- Performance Testing & Debugging for black boxes
- Fault Tolerance and Reliability Testing
- Security Testing (Mobile Security)
- Markov Decision Processes (MDPs)
 - L^* -Based Learning Algorithms
 - As seen in VU Quality Assurance
→ (Can be taken next summer 😊)

and many more applications (cited hundreds of times!!)



[International Symposium on Formal Methods](#)

→ FM 2019: **Formal Methods – The Next 30 Years** pp 651–669 | [Cite as](#)

[Home](#) > [Formal Methods – The Next 30 Years](#) > [Conference paper](#)

L^* -Based Learning of Markov Decision Processes

[Martin Tappler](#) , [Bernhard K. Aichernig](#), [Giovanni Bacci](#), [Maria Eichlseder](#) & [Kim G. Larsen](#)

Conference paper | [First Online: 23 September 2019](#)

1654 Accesses | 15 Citations | 3 Altmetric

Part of the [Lecture Notes in Computer Science](#) book series (LNPS, volume 11800)

Questions?

References

- [1] E.M. Gold: Complexity of Automaton Identification from Given Data. *Information and Control* **37** (1978) 302–320.
- [2] D. Angluin: Identifying Languages from Stochastic Examples. Internal Report YALEU/DCS/RR-614 (1988).
- [3] F.J. Maryanski and T.L. Booth: Inference of Finite-State Probabilistic Grammars. *IEEE Transactions on Computers* **C26** (1977) 521–536.
- [4] A. van der Mude and A. Walker: On the Inference of Stochastic Regular Grammars. *Information and Control* **38** (1978) 310–329.
- [5] A.W. Smith and D. Zipser: Learning Sequential Structure with the Real-Time Recurrent Learning Algorithm. *International Journal of Neural Systems* **1** (1989) 125–131.
- [6] J.B. Pollack: The Induction of Dynamical Recognizers. *Machine Learning* **7** (1991) 227–252.
- [7] C.L. Giles: Learning and Extracting Finite State Automata with Second Order Recurrent Neural Networks. *Neural Computation* **4** (1992) 393–405.
- [8] R.L. Wartous and G.M. Kuhn: Induction of Finite-state Languages Using Second-Order Recurrent Networks. *Neural Computation* **4** (1992) 406–414.
- [9] M.A. Castaño, F. Casacuberta, E. Vidal: Simulation of Stochastic Regular Grammars through Simple Recurrent Networks, in: *New Trends in Neural Computation* (Eds. J. Mira, J. Cabestany and A. Prieto). Springer Verlag, Lecture Notes in Computer Science **686** (1993) 210–215.
- [10] A. Stolcke and S. Omohundro: Hidden Markov Model Induction by Bayesian Model Merging. To appear in: *Advances in Neural Information Processing Systems 5* (C.L. Giles, S.J. Hanson and J.D. Cowan eds.) Morgan Kaufman, Menlo Park, California (1993).
- [11] J. Oncina and P. García: Inferring Regular Languages in Polynomial Time, in: *Pattern Recognition and Image Analysis* (N. Pérez de la Blanca, A. Sanfeliu and E. Vidal eds.) World Scientific (1992).
- [12] K.S. Fu: *Syntactic Pattern Recognition and Applications*. Prentice Hall, Englewood Cliffs, N.J. (1982).
- [13] J.E. Hopcroft and J.D. Ullman: *Introduction to Automata Theory, Languages and Computation*. Addison Wesley, Reading, Massachusetts (1979).
- [14] W. Hoeffding: Probability inequalities for sums of bounded random variables. *American Statistical Association Journal* **58** (1963) 13–30.
- [15] W. Feller: *An introduction to probability theory and its applications*. John Wiley and Sons, New York (1950)
- [16] A.S. Reber: Implicit Learning of Artificial Grammars. *Journal of Verbal Learning and Verbal Behaviour* **6** (1967) 855–863.